

softmax 回归从零实现—逐行精讲

学习笔记

2026 年 8 月 25 日

整体框架：一条流水线

从零实现的所有代码，本质是下面这条流水线的每一环：

数据加载 → 初始化参数 → 定义 softmax → 定义模型 → 定义损失 → 定义精度 → 训练循环

其中只有三块是 *softmax* 特有的**新知识**（核心），其余和线性回归几乎一样：

- （核心）定义 **softmax**：把分数变成概率（softmax 的核心）
- （核心）交叉熵损失： $y_hat[\text{range}(n), y]$ 取正确类别概率（fancy indexing）
- （核心）分类精度：argmax 取预测类别 + 比较统计（分类独有，回归没有）
- 其余（加载数据、初始化、训练循环骨架）= 线性回归照搬

1 加载数据与初始化参数（简单，线性回归照搬）

1.1 加载数据

```
1 batch_size = 256
2 train_iter, test_iter = d2l.load_data_fashion_mnist(batch_size)
```

Fashion-MNIST：28 × 28 灰度图，10 类衣服。这一行加载训练集和测试集，切成 256 一批。（细节见 3.5 节，这里直接用。）

1.2 初始化参数（有一个关键区别）

```
1 num_inputs = 784
2 num_outputs = 10
3 W = torch.normal(0, 0.01, size=(num_inputs, num_outputs), requires_grad=True)
4 b = torch.zeros(num_outputs, requires_grad=True)
```

- 784 = 28 × 28：每张图展平成一个 784 维向量（把每个像素当一个特征）
- 10：类别数

- **关键区别：**线性回归的 W 是 $(d,)$ 向量；softmax 的 W 是 $(784, 10)$ 矩阵——每个输入特征连到每个类别一个权重
- b 是 $(10,)$ 向量——每个类别一个偏置

2 定义 softmax（核心）

2.1 回顾公式

$$\text{softmax}(X)_{ij} = \frac{\exp(X_{ij})}{\sum_k \exp(X_{ik})}$$

分子：每个元素取指数；分母：第 i 行（一个样本）所有指数的和。

2.2 三步实现的代码

```
1 def softmax(X):
2     X_exp = torch.exp(X)           # 1. 每项取 exp
3     partition = X_exp.sum(1, keepdim=True) # 2. 每行求和（规范化常数）
4     return X_exp / partition       # 3. 除以行和（广播）
```

2.3 逐行精讲

第 1 行 `X_exp = torch.exp(X)`：对每个元素取指数，保证非负。

第 2 行 `partition = X_exp.sum(1, keepdim=True)`：

- `sum(1)`：沿第 1 轴（列）求和，即对每一行求和（每个样本一行）
- `keepdim=True`：关键！保持形状为 $(n, 1)$ ，而不是塌缩成 $(n,)$
- 为什么必须 `keepdim`：下面第 3 行要做 $(n, C)/(n, 1)$ 的广播。若没有 `keepdim`，得到 $(n,)$ ，右对齐广播会错位（ (n, C) 与 $(n,)$ 的最后一维对不上）

第 3 行 `return X_exp / partition`： $(n, C)/(n, 1)$ 触发广播——每一行除以自己那一行的和，结果每行和为 1。

2.4 验证性质

对任意输入 X ，输出每行都是非负数且和为 1（合法概率分布）。

3 定义模型 net（重点）

```
1 def net(X):
2     return softmax(torch.matmul(X.reshape((-1, W.shape[0])), W) + b)
```

从内到外拆解：

- (1) `X.reshape((-1, W.shape[0]))`: 把每张 28×28 图展平成 784 维向量。-1 表示”自动推断 batch 大小”，`W.shape[0] = 784`。
- (2) `torch.matmul(..., W)`: $X@W$ ，得到 $(n, 10)$ 的分数 (logits)。
- (3) `+ b`: 加偏置 (广播)。
- (4) `softmax(...)`: 把分数变成概率。

作用：输入一张图，输出 10 个概率。与线性回归的 $X@w+b$ 结构完全相同，只是多包了一层 softmax。

4 交叉熵损失（核心）

4.1 核心公式

$$L = -\log \hat{y}_y \quad (\text{取“正确类别”的概率，取负对数})$$

4.2 代码

```
1 def cross_entropy(y_hat, y):
2     return - torch.log(y_hat[range(len(y_hat)), y])
```

4.3 逐行精讲（重点在 fancy indexing）

`y_hat[range(len(y_hat)), y]` 这一句是整节最理解的地方：

- `y_hat`: 形状 $(n, 10)$ 的预测概率矩阵，每一行是一个样本的 10 个概率
- `y`: 形状 $(n,)$ 的真实标签（每个样本的正确类别编号，如 $[0, 2]$ ）
- `range(len(y_hat)) = [0, 1, ..., n - 1]`: 行索引
- `y_hat[行索引, 列索引]`: 这是 **fancy indexing**（高级索引），按位置配对取出元素：
 - 第 0 行取第 `y[0]` 列（第 0 个样本的正确类别概率）
 - 第 1 行取第 `y[1]` 列（第 1 个样本的正确类别概率）
 - ...

例子：`y = [0, 2]`，`y_hat = [[0.1, 0.3, 0.6], [0.3, 0.2, 0.5]]`，则 `y_hat[[0, 1], y] = [0.1, 0.5]`——分别取了第 0 行第 0 列、第 1 行第 2 列。

为什么要用这个技巧：不用 Python 的 for 循环（低效），一行向量化地”挑出每个样本正确类别的概率”。

`-torch.log(...)`: 对挑出的概率取负对数——这就是”完整形式 $-\sum_k y_k \log \hat{y}_k$ ”在 one-hot 下的化简（其他类别 $y_k = 0$ 全消失）。

5 分类精度 accuracy ((核心) 重点, 分类独有)

5.1 代码

```
1 def accuracy(y_hat, y):
2     if len(y_hat.shape) > 1 and y_hat.shape[1] > 1:
3         y_hat = y_hat.argmax(axis=1) # 取每行最大值的索引 = 预测类别
4         cmp = y_hat.type(y.dtype) == y # 与真实标签比较
5         return float(cmp.type(y.dtype).sum()) # 正确个数
```

5.2 逐行精讲

第 1-2 行 `y_hat.argmax(axis=1)`: 对每一行 (每个样本) 取概率最大的类别的索引, 作为硬预测 (hard prediction)。例如某行 `[0.1, 0.3, 0.6]`, `argmax` 得 2。

第 3 行 `y_hat.type(y.dtype) == y`: 预测类别和真实标签逐元素比较, 得到 0/1 张量 (相等为 1)。`type(y.dtype)` 先统一数据类型 (== 对类型敏感)。

第 4 行 `.sum()`: 求和 = 正确预测的个数。除以总数即精度。

为什么回归没有这一块: 回归输出连续值, 没有“预测类别对没对”; 分类输出离散类别, 需要“对错计数”。这是分类特有的评估指标。

6 训练循环 ((核心) 重点, 骨架与线性回归一致)

6.1 核心循环 (只保留主线)

```
1 for X, y in train_iter:
2     y_hat = net(X) # 前向
3     l = loss(y_hat, y) # 损失
4     l.sum().backward() # 反向 (定制版用 sum)
5     updater(X.shape[0]) # 更新参数 (内部 d2l.sgd)
```

和线性回归的训练循环一字不差: 前向 → 损失 → backward → 更新。唯一的差别是把 MSE 换成交叉熵、把 `linreg` 换成 `net` (多包一层 softmax)。

6.2 updater (更新函数)

```
1 lr = 0.1
2 def updater(batch_size):
3     return d2l.sgd([W, b], lr, batch_size)
```

复用线性回归的手写 SGD (no_grad + 原地 -= + 除以 batch_size 求平均)。学习率 0.1。

7 可跳过的辅助代码 (了解即可)

以下代码是工程脚手架, 不影响理解 softmax 原理, 看懂“它在干什么”即可, 不必精读:

7.1 Accumulator（累加器）

```
1 class Accumulator:
2     def __init__(self, n):
3         self.data = [0.0] * n
4     def add(self, *args):
5         self.data = [a + float(b) for a, b in zip(self.data, args)]
```

作用：训练中累加” 损失总和、正确数、样本数” 等统计量。就是几个计数器的容器。

7.2 evaluate__accuracy（评估精度）

```
1 def evaluate_accuracy(net, data_iter):
2     with torch.no_grad():          # 评估不需要梯度，省内存
3         for X, y in data_iter:
4             metric.add(accuracy(net(X), y), y.numel())
5     return metric[0] / metric[1] # 正确数 / 总数
```

作用：在测试集上跑一遍，统计正确率。torch.no_grad() 表示” 只看结果、不求梯度”。

7.3 Animator（画图工具）

纯绘图工具，把训练曲线画成动画，完全可跳过。

8 一句话总结

从零实现 softmax 的全部新知识就三件事：

1. **softmax**: $\exp \rightarrow$ 行求和 (keepdim) $\rightarrow$ 除，把分数变概率
2. **交叉熵**: $y_hat[range(n), y]$ 取正确类别概率，取负对数
3. **精度**: argmax 取预测类别，和真实标签比较计数

其余（加载数据、初始化、训练循环）都是线性回归的照搬。掌握这三块，从零实现就通了。